

Lending Interaction Wings to Recommender Systems with Conversational Agents

Jiarui Jin¹, Xianyu Chen¹, Fanghua Ye², Mengyue Yang², Yue Feng²,
Weinan Zhang¹, Yong Yu¹, Jun Wang²

¹Shanghai Jiao Tong University, ²University College London



SHANGHAI JIAO TONG
UNIVERSITY



Content

- Problem Background
 - Conversational Recommender Systems
 - Current Challenges
- Architecture
 - Overall Framework
 - Online-Checking Module
 - Offline-Training Module
- Experiment
- Conclusion

Content

- **Problem Background**
 - Conversational Recommender Systems
 - Current Challenges
- **Architecture**
 - Overall Framework
 - Online-Checking Module
 - Offline-Training Module
- **Experiment**
- **Conclusion**

Conversational Recommender Systems

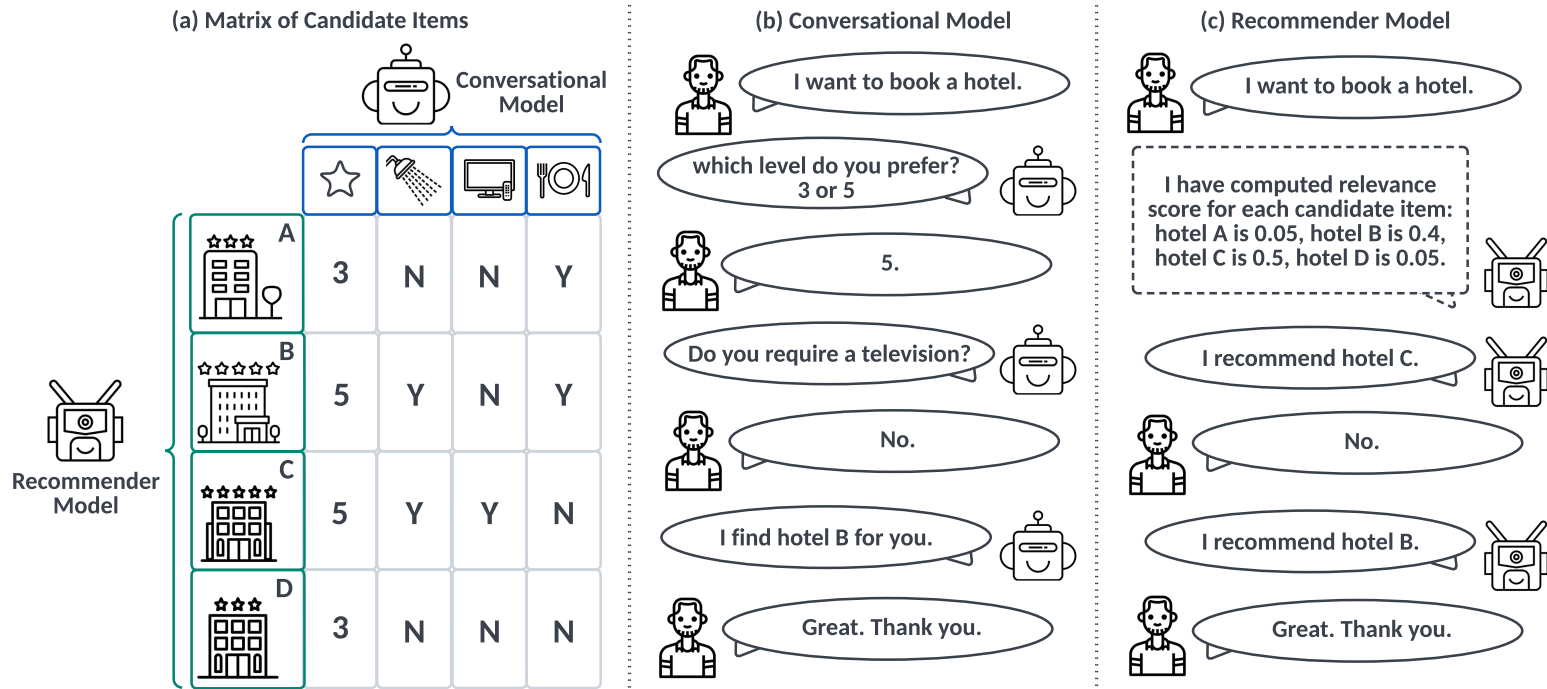


Figure: Comparisons between conversational model and (classical) recommender model.

Most currently deployed recommendation platforms are based on classical recommender models (**assigning estimated score to each item** according to historical data of users).

Conversational Recommender Systems

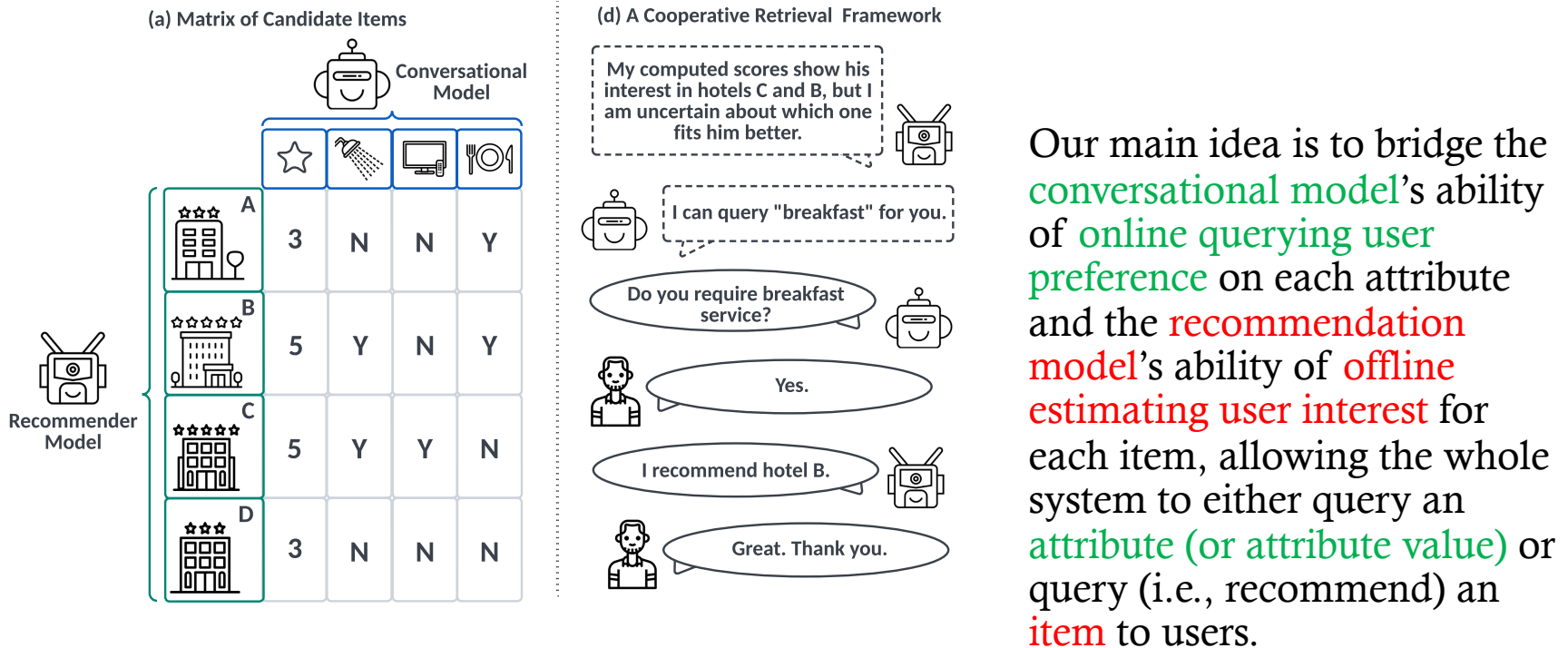


Figure: Cooperation between conversational model and (classical) recommender model.

Our main idea is to bridge the **conversational model's** ability of **online querying user preference** on each attribute and the **recommendation model's** ability of **offline estimating user interest** for each item, allowing the whole system to either query an **attribute (or attribute value)** or query (i.e., recommend) an **item** to users.

Current Challenges

- [How to determine which one attribute (or attribute value) or one item to query]
- The key technique is how to jointly measure the reward (information gain) of querying **attribute values** and **items**.

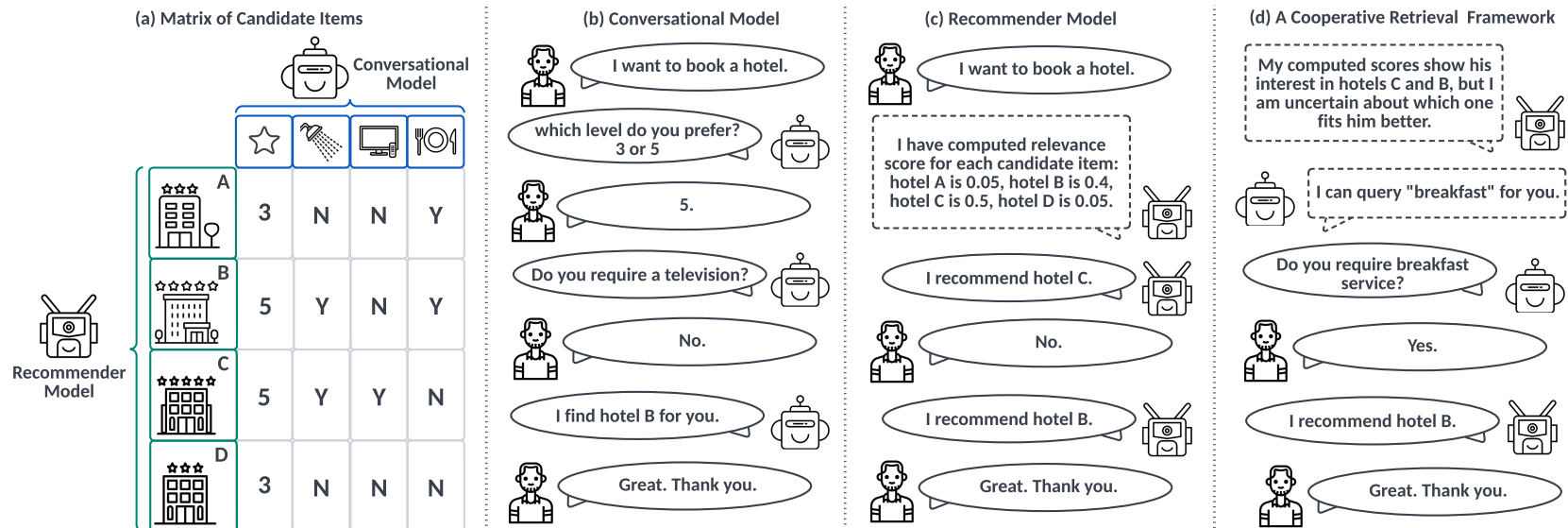


Figure: Cooperation between conversational model and (classical) recommender model.

Current Challenges

- [How to design the interaction mechanism with users]
- Previous literature such as [1][2] introduce **conversational module** together with **recommender** module, and jointly train them via reinforcement learning.

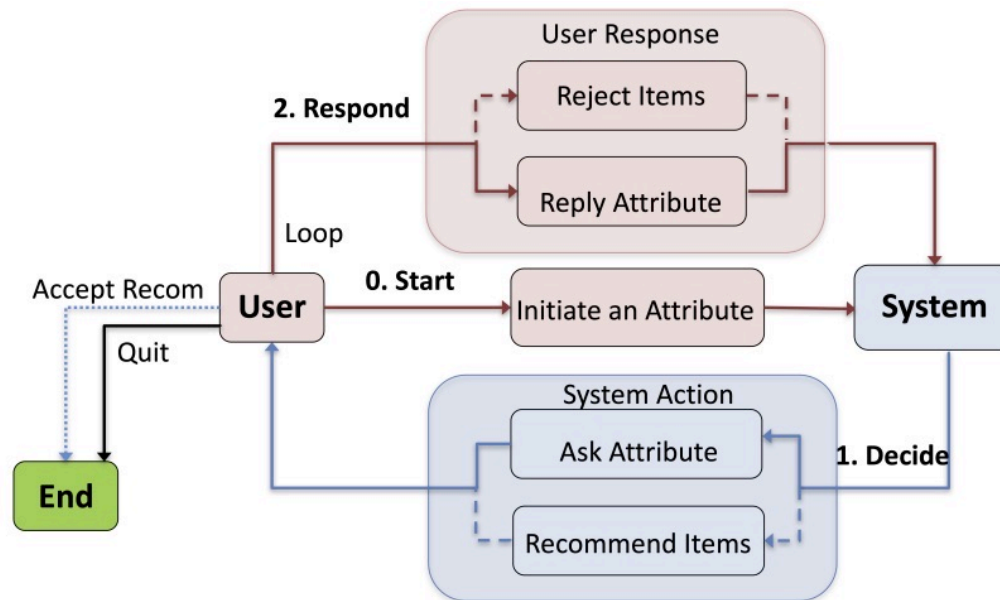


Figure: The workflow of multi-round conversational recommendation scenario.

Ref: Wenqiang Lei, et al. Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems. WSDM, 2020.

Ref: Yueming Sun, et al. Conversational Recommender System. SIGIR, 2018.

Content

- Problem Background
 - Conversational Recommender Systems
 - Current Challenges
- **Architecture**
 - Overall Framework
 - Online-Checking Module
 - Offline-Training Module
- Experiment
- Conclusion

Overall Framework

Bridge recommender systems and conversational agents through an offline-training and online-checking framework, only requiring **estimated scores** from recommender systems (offline-training) and **APIs** from conversational agent (online checking).

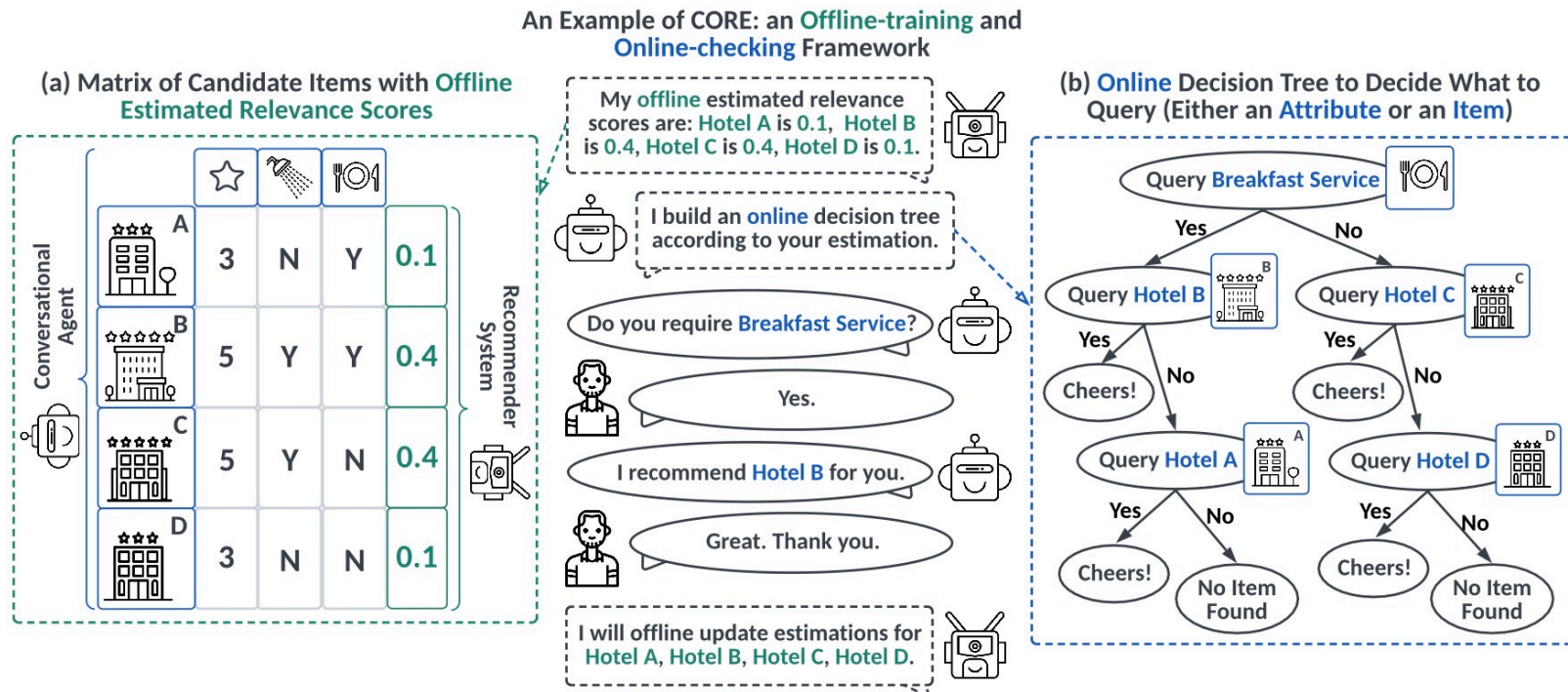


Figure: Overview of our offline-training and online-checking framework.

Online Decision Tree

- Given estimated **relevance scores** from an **offline-trained recommender system**, our certainty gain is defined as **how many relevance scores have been checked by querying an item or an attribute value**.

Definition 1 (Uncertainty and Certainty Gain). For the k -th turn, we define uncertainty, denoted as U_k , to measure how many estimated relevance scores are still unchecked, i.e.,

$$U_k := \text{SUM}(\{\Psi_{\text{RE}}(v_m) | v_m \in \mathcal{V}_k\}), \quad (1)$$

where $\Psi_{\text{RE}}(v_m)$ ¹ outputs the estimated relevance score for item v_m , $\mathcal{V}_k$ is the set of all the unchecked items after k interactions, and $\mathcal{V}_k$ is initialized as $\mathcal{V}_0 = \mathcal{V}$. Then, the certainty gain of k -th interaction is defined as $\Delta U_k := U_{k-1} - U_k$, i.e., how many relevance scores are checked at the k -th turn. Since our goal is to find **a target item** if $\Psi_{\text{CD}}(\cdot)$ successfully finds one at the k -th turn, then we set all the items checked, namely $U_k = 0$.

In this regard, our conversational agent is to minimize U_k at each k -th turn via removing those checked items from $\mathcal{V}_k$. Considering that online updating $\Psi_{\text{RE}}(\cdot)$ is infeasible in practice due to the high latency and computation costs, the objective of $\Psi_{\text{CD}}(\cdot)$ can be expressed as:

$$\min_{\Psi_{\text{RE}}^*} K, \text{ s.t., } U_K = 0, \quad (2)$$

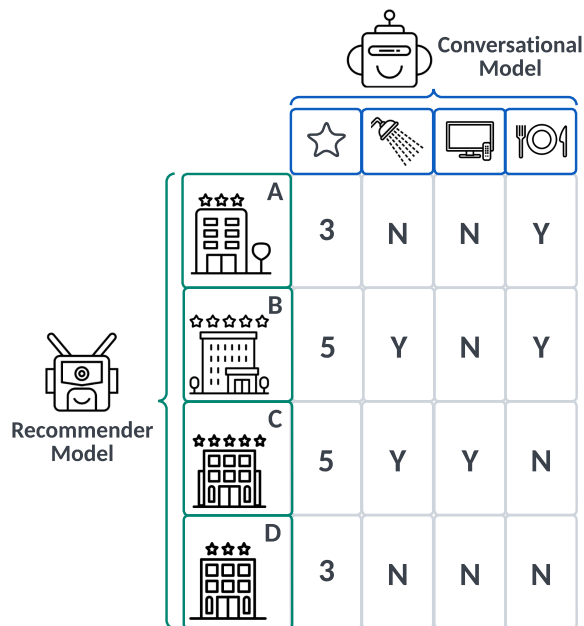
where K is the number of turns, and Ψ_{RE}^* means that the recommender system is frozen. To this end, the design of our conversational agent could be organized as an uncertainty minimization problem.

Online Checking Module

- The resulting online decision tree algorithm is to greedily choose an attribute (value) or an item that can maximize the certainty gain:

$$a_{\text{query}} = \arg \max_{a \in \mathcal{V}_{k-1} \cup \mathcal{X}_{k-1}} \Psi_{\text{CG}}(\text{query}(a)),$$

(a) Matrix of Candidate Items



	★	🚿	📺	🍴
A ★★★	3	N	N	Y
B ★★★★	5	Y	N	Y
C ★★★★★	5	Y	Y	N
D ★★★	3	N	N	N

- The formulation of querying **an attribute** can be written as:

$$\Psi_{\text{CG}}(\text{query}(a)) = \sum_{w_a \in \mathcal{W}_a} \left(\Psi_{\text{CG}}(w_a = w_a^*) \cdot \Pr(w_a = w_a^*) \right),$$

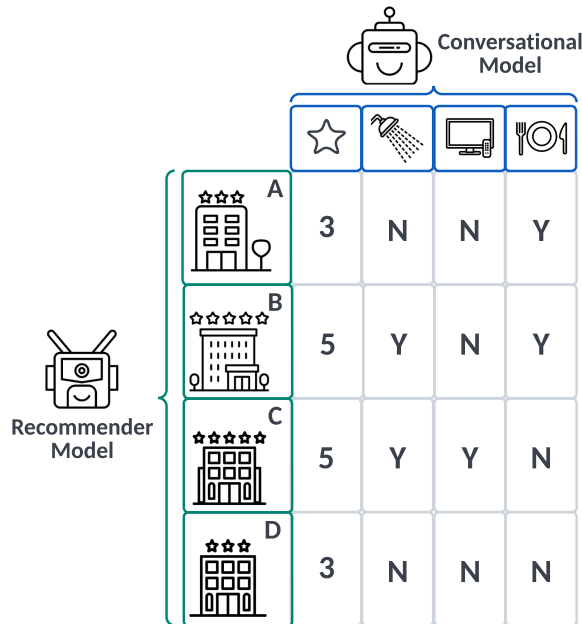
where $w_a = w_a^*$ means that when querying a , the user's answer (represented by w_a^* is w_a).

Online Checking Module

- The resulting online decision tree algorithm is to greedily choose an attribute (value) or an item that can maximize the certainty gain:

$$a_{\text{query}} = \arg \max_{a \in \mathcal{V}_{k-1} \cup \mathcal{X}_{k-1}} \Psi_{\text{CG}}(\text{query}(a)),$$

(a) Matrix of Candidate Items



	★	🚿	📺	🍴
A	3	N	N	Y
B	5	Y	N	Y
C	5	Y	Y	N
D	3	N	N	N

- The formulation of querying **an item** is

$$\begin{aligned} \Psi_{\text{CG}}(\text{query}(a)) &= \Psi_{\text{CG}}(a \in \mathcal{V}^*) \cdot \Pr(a \in \mathcal{V}^*) + \Psi_{\text{CG}}(a \notin \mathcal{V}^*) \cdot \Pr(a \notin \mathcal{V}^*) \\ &= \left(\sum_{v_m \in \mathcal{V}_{k-1}} \Psi_{\text{RE}}(v_m) \right) \cdot \Pr(a \in \mathcal{V}^*) + \Psi_{\text{RE}}(a) \cdot \Pr(a \notin \mathcal{V}^*), \end{aligned}$$

where $a \in \mathcal{V}^*$ means that queried a is **a target item**.

Online Checking Module

We reveal the connection between querying **attributes** (i.e., querying **attribute IDs**) and querying attribute values in the following proposition.

Proposition 1. *For any attribute $x \in \mathcal{X}_{k-1}$, $\Psi_{\text{CG}}(\text{query}(x)) \geq \Psi_{\text{CG}}(\text{query}(x) = w_x)$ holds for all the possible $w_x \in \mathcal{W}_x$.*

In the context of querying attribute values, we further reveal what kind of attribute value is an ideal one in the following proposition.

Proposition 2. *In the context of querying attribute values, an ideal choice is always the one that can partition all the unchecked relevance scores into two equal parts (i.e., the ideal $w_x \in \mathcal{W}_x, x \in \mathcal{X}_{k-1}$ is the one that makes $\Psi_{\text{CG}}(w_x = w_x^*) = \text{SUM}(\{\Psi_{\text{RE}}(v_m) | v_m \in \mathcal{V}_{k-1}\})/2$ hold), if it is achievable. And the certainty gain in this case is $\Psi_{\text{CG}}(\text{query}(x) = w_x) = \text{SUM}(\{\Psi_{\text{RE}}(v_m) | v_m \in \mathcal{V}_{k-1}\})/2$.*

To get rid of the impact of the recommender system, we introduce a cold-start setting, where $\Psi_{\text{RE}}(v_m) = 1/M$ holds for any $v_m \in \mathcal{V}$.

Lemma 1. *In the context of querying attribute values, suppose that $\Psi_{\text{RE}}(v_m) = 1/M$ holds for any $v_m \in \mathcal{V}$, then the expected number of turns (denoted as $\hat{K}$) is bounded by $\log_2^{M+1} \leq \hat{K} \leq (M+1)/2$.*

Convolutional recommender systems (purely recommending an item at each turn) can be regarded as a **special case** of our framework.

Online Checking Module

- We notice that the above formulations of either querying attributes or querying attribute values, **does not consider the dependence among attributes** (e.g., as Figure 1(a) shows, attribute **Hotel Level** can largely determine attribute **Shower Service**).

$$\Psi_{\text{CG}}^{\text{D}}(\text{query}(a)) = \sum_{a' \in \mathcal{X}_{k-1}} \left(\Psi_{\text{CG}}(\text{query}(a')) \cdot \Pr(\text{query}(a') | \text{query}(a)) \right),$$

$\Pr(\text{query}(a') | \text{query}(a))$ measures the probability of the user preference on a determining the user preference on a' .

$$\Psi_{\text{CG}}^{\text{D}}(\text{query}(a)) = \sum_{w_a \in \mathcal{W}_a} \left(\Psi_{\text{CG}}^{\text{D}}(w_a = w_a^*) \cdot \Pr(w_a = w_a^*) \right),$$

where we define $\Psi_{\text{CG}}^{\text{D}}(w_a = w_a^*)$ as:

$$\Psi_{\text{CG}}^{\text{D}}(w_a = w_a^*) = \sum_{a' \in \mathcal{A}} \sum_{w_{a'} \in \mathcal{W}_{a'}} \left(\Psi_{\text{CG}}(w_{a'} = w_{a'}^*) \cdot \Pr(w_{a'} = w_{a'}^* | w_a = w_a^*) \right),$$

where $\Pr(w_{a'} = w_{a'}^* | w_a = w_a^*)$ measures the probability of how likely getting the user preference on attribute a determinates the user's preference on other attributes a' .

Offline Training Module

- Our paper lists two separate configurations in terms of querying attributes: one is directly querying **attribute IDs** (e.g., querying a user what **level of hotel** she prefers), while the other one is query **attribute values** (e.g., querying a user whether she prefers hotels with **level 3**).

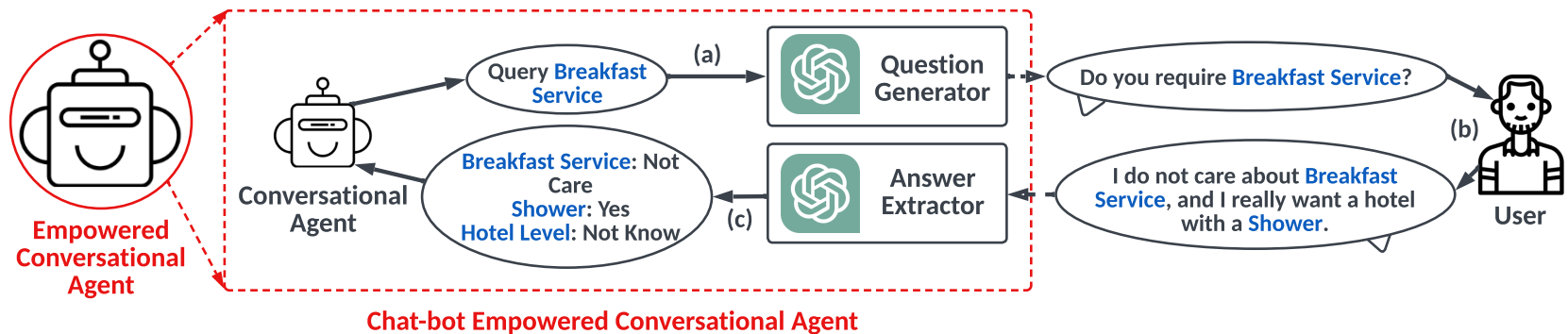
Algorithm 1 CORE for Querying Items and Attributes

Input: A recommender system $\Psi_{\text{RE}}(\cdot)$, an item set $\mathcal{V}$, an attribute set $\mathcal{X}$, an offline dataset $\mathcal{D}$.

Output: Updated recommender system $\Psi_{\text{RE}}(\cdot)$, up-to-date dataset $\mathcal{D}$.

- 1: Train $\Psi_{\text{RE}}(\cdot)$ on $\mathcal{D}$. ▷ Offline-Training
 - 2: **for** each session (i.e., the given user) **do**
 - 3: Initialize $k = 1$ and $\mathcal{V}_0 = \mathcal{V}$, $\mathcal{X}_0 = \mathcal{X}$.
 - 4: **repeat**
 - 5: Compute a_{query} following Eq. (3) for querying items and attributes or following Eq. (10) for querying items and attribute values. ▷ Online-Checking
 - 6: Query a_{query} to the user and receive the answer. ▷ Online-Checking
 - 7: Generate $\mathcal{V}_k$ and $\mathcal{X}_k$ from $\mathcal{V}_{k-1}$ and $\mathcal{X}_{k-1}$. ▷ Online-Checking
 - 8: Go to next turn: $k \leftarrow k + 1$.
 - 9: **until** Querying a target item or $k > K_{\text{MAX}}$ where K_{MAX} is the maximal number of turns.
 - 10: Collect session data and add to $\mathcal{D}$.
 - 11: **end for**
 - 12: Update $\Psi_{\text{RE}}(\cdot)$ using data in $\mathcal{D}$. ▷ Offline-Training
-

Prompt Design for Extractor and Generator



- Why use LLM here:
 - LLM is a powerful language model for understanding and generating conversations.
 - LLM can encode some **latent features** from conversations to guide the decision-making of the conversational agent.

Prompt Design for Extractor

```
"""
prompt = f"""
You will be provided with text delimited by triple quotes.
If you can infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: [user preference]
Attribute names include Breakfast Service, Hotel Level, and Shower.
User preference includes Yes to denote the positive preference, No to denote the
negative preference, and Not Care to denote the user does not care.
If you can not infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: Not Know
'''{text}'''
"""

response = get_completion(prompt)
print(response)
```

The following is the corresponding output by the LLM.

```
Breakfast Service: Not Care
Hotel Level: Not Know
Shower: Yes
```

Prompt Design for Extractor

```
# Large language model as latent feature extractor.
text = f"""
I am a student, I do not care about breakfast service, and I really want a hotel with a shower.
"""

prompt = f"""
You will be provided with text delimited by triple quotes.
If you can infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: [user preference]
Attribute names include Breakfast Service, Hotel Level, Price, and Shower.
User preference includes Yes to denote the positive preference, No to denote the
negative preference, and Not Care to denote the user does not care.
If you can infer the user has a low budget, please make Price: Low.
If you can not infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: Not Know
'''{text}'''
"""

response = get_completion(prompt)
print(response)
```

The following is the corresponding output by the LLM.

```
Breakfast Service: Not Care
Hotel Level: Not Know
Price: Low
Shower: Yes
```

Prompt Design for Generator

```
# Large language model as question generator.
text = f"""
Attribute, Hotel Level is 5, Hotel
"""

prompt = f"""
You will be provided with text delimited by triple quotes.
If it starts with the word "item", it denotes an item \
then you should generate a text to recommend the item to the user.
Otherwise, it denotes an attribute \
then you should generate a text to query the user's preference on the attribute.
You should be gentle.
‘‘{text}’’
"""

response = get_completion(prompt)
print(response)
```

The following is the corresponding output by the LLM.

Excuse me, may I ask for your preference on hotel level? Would you prefer a 5-star hotel or are you open to other options?

Content

- Problem Background
 - Conversational Recommender Systems
 - Current Challenges
- Architecture
 - Overall Framework
 - Online-Checking Module
 - Offline-Training Module
- **Experiment**
- Conclusion

Experiment

Table A1: Results comparison in the context of querying attributes and items on tabular datasets.

$\Psi_{RE}(\cdot)$	$\Psi_{CO}(\cdot)$	Amazon				LastFM				Yelp			
		T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5
COLD START	AG	6.47	0.12	7.83	0.23	6.77	0.05	8.32	0.14	6.65	0.08	8.29	0.13
	ME	3.04	0.98	5.00	1.00	3.00	1.00	5.00	1.00	3.00	1.00	5.00	1.00
	CORE	2.88	1.00	2.87	1.00	2.73	1.00	2.75	1.00	2.92	1.00	2.94	1.00
	CORE _D ⁺	2.84	1.00	2.86	1.00	2.74	1.00	2.73	1.00	2.90	1.00	2.91	1.00
FM	AG	2.76	0.74	2.97	0.83	4.14	0.52	4.67	0.64	3.29	0.70	3.39	0.81
	CRM	3.07	0.98	3.37	0.83	2.98	0.99	3.43	1.00	3.08	0.98	3.12	0.96
	EAR	2.98	0.99	3.13	1.00	3.02	1.00	3.51	1.00	2.94	1.00	3.02	0.99
	CORE	2.17	1.00	2.16	1.00	2.06	1.00	2.07	1.00	2.09	1.00	2.10	1.00
	CORE _D ⁺	2.14	1.00	2.14	1.00	2.05	1.00	2.05	1.00	2.10	1.00	2.08	1.00

[ME] always generates a query in the attribute level with the maximum entropy principle.

[AG] always queries an item with the highest relevance score at each turn, which is equivalent to solely using recommender systems.

[CRM] [Yueming Sun, et al.] integrates the conversational component and the recommender component by feeding the belief tracker results to the recommendation method.

[EAR] [Wenqiang Lei, et al.] consists of three stages, i.e., the estimation stage, the action stage, and the reflection stage.

[FM] [Steffen Rendle] considers the second-order interactions among attributes (i.e., feature fields).

[DEEP FM] [Huifeng Guo, et al.] combines an FM component and a neural network component together to produce the final prediction.

[PNN] [Yanru Qu, et al.] includes an embedding layer to learn a representation of the categorical data and a representation of user interests from historical behaviors.

[CORE] is our method but not consider the dependence among attributes, and CORE_D⁺ considers.

Ref: Wenqiang Lei, et al. Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems. WSDM, 2020.

Ref: Yueming Sun, et al. Conversational Recommender System. SIGIR, 2018.

Ref: Steffen Rendle. Factorization Machines. ICDM, 2010.

Ref: Huifeng Guo, et al. DeepFM: A Factorization-machine based Neural Network for CTR Prediction. IJCAI, 2017.

Ref: Yanru Qu, et al. Product-based Neural Networks for User Response Prediction. ICDM, 2016.

Experiment

Table 2: Results comparison of querying attribute values on tabular datasets. See Table A2 for the full version.

$\Psi_{RE}(\cdot)$	$\Psi_{CO}(\cdot)$	Amazon				LastFM				Yelp			
		T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5
COLD START	AG	6.47	0.12	7.83	0.23	6.77	0.05	8.32	0.14	6.65	0.08	8.29	0.13
	ME	6.50	0.12	8.34	0.16	6.84	0.04	8.56	0.11	6.40	0.15	8.18	0.20
	CORE	6.02	0.25	6.18	0.65	5.84	0.29	5.72	0.74	5.25	0.19	6.23	0.65
	CORE _D ⁺	6.00	0.26	6.01	0.67	5.79	0.30	5.70	0.75	5.02	0.21	6.12	0.68
FM	AG	2.76	0.74	2.97	0.83	4.14	0.52	4.67	0.64	3.29	0.70	3.39	0.81
	CRM	4.58	0.28	6.42	0.38	4.23	0.34	5.87	0.63	4.12	0.25	6.01	0.69
	EAR	4.13	0.32	6.32	0.42	4.02	0.38	5.45	0.67	4.10	0.28	5.95	0.72
	CORE	3.26	0.83	3.19	0.99	3.79	0.72	3.50	0.99	3.14	0.84	3.20	0.99
	CORE _D ⁺	3.16	0.85	3.22	1.00	3.75	0.74	3.53	1.00	3.10	0.85	3.23	1.00
DEEP FM	AG	3.07	0.71	3.27	0.82	3.50	0.68	3.84	0.79	3.09	0.74	3.11	0.88
	CRM	4.51	0.29	6.32	0.40	4.18	0.38	5.88	0.63	4.11	0.23	6.02	0.71
	EAR	4.47	0.30	6.35	0.43	4.01	0.37	5.43	0.69	4.01	0.32	5.74	0.75
	CORE	3.23	0.85	3.22	0.99	3.47	0.81	3.34	1.00	2.98	0.93	3.11	1.00
PNN	AG	3.02	0.74	3.10	0.87	3.44	0.67	3.53	0.84	2.83	0.77	2.82	0.91
	CORE	3.01	0.88	3.04	0.99	3.10	0.87	3.20	0.99	2.75	0.88	2.76	1.00

Ref: Wenqiang Lei, et al. Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems. WSDM, 2020.

Ref: Yueming Sun, et al. Conversational Recommender System. SIGIR, 2018.

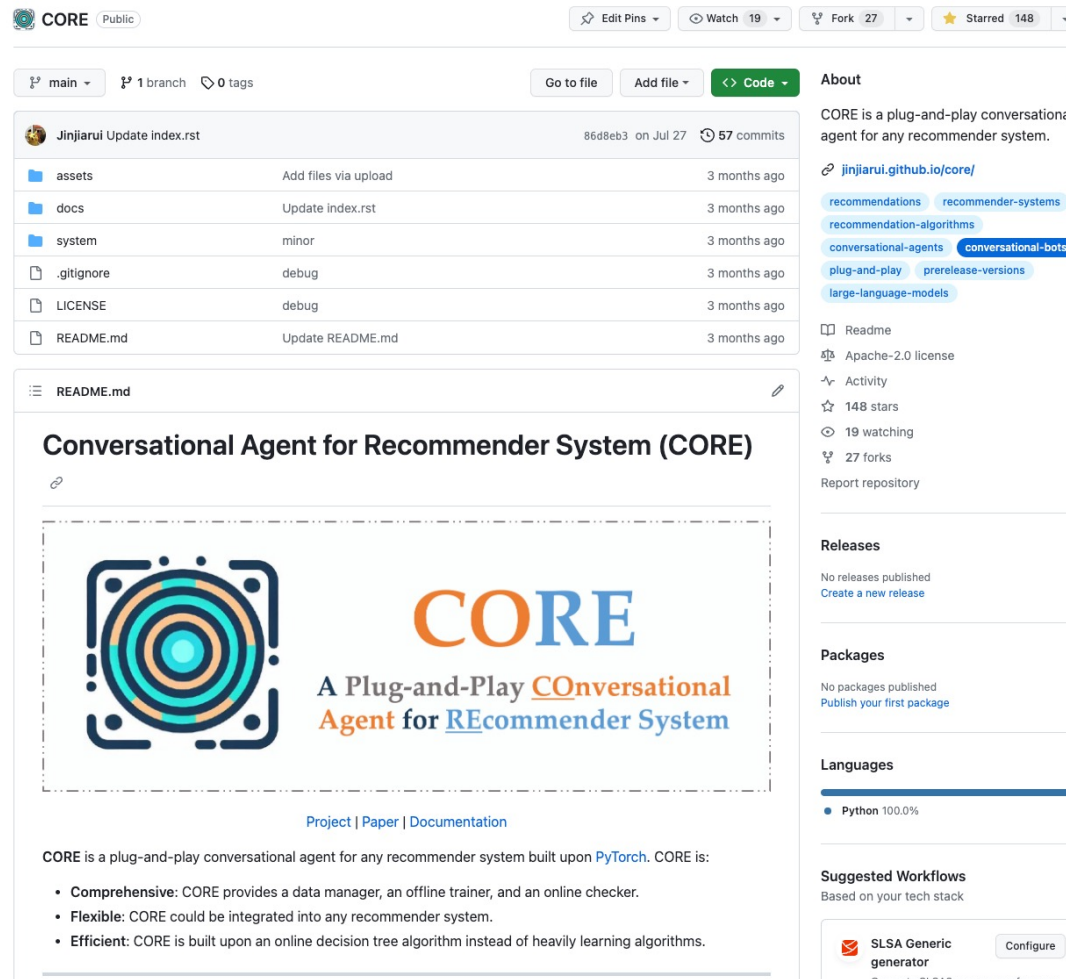
Ref: Steffen Rendle. Factorization Machines. ICDM, 2010.

Ref: Huifeng Guo, et al. DeepFM: A Factorization-machine based Neural Network for CTR Prediction. IJCAI, 2017.

Ref: Yanru Qu, et al. Product-based Neural Networks for User Response Prediction. ICDM, 2016.

Open-Source System

- Now we support **top-N** recommendations for items and attribute values.



The screenshot shows the GitHub repository for 'CORE' by Jinjia Rui. The repository is public and has 19 watchers, 27 forks, and 148 stars. The main branch is 'main'. The repository contains files like 'assets', 'docs', 'system', '.gitignore', 'LICENSE', and 'README.md'. The 'README.md' file is open, showing the project's title 'Conversational Agent for Recommender System (CORE)' and a logo. The logo features a stylized camera icon with concentric circles and the text 'CORE A Plug-and-Play CONversational Agent for REcommender System'. Below the logo, there are links for 'Project | Paper | Documentation'. The text describes CORE as a plug-and-play conversational agent for any recommender system built upon PyTorch. It lists three features: Comprehensive (data manager, offline trainer, online checker), Flexible (integrates into any recommender system), and Efficient (built upon an online decision tree algorithm instead of heavily learning algorithms). The right sidebar shows the 'About' section, which describes CORE as a plug-and-play conversational agent for any recommender system. It also lists tags like 'recommendations', 'recommender-systems', 'recommendation-algorithms', 'conversational-agents', 'conversational-bots', 'plug-and-play', 'prerelease-versions', and 'large-language-models'. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows Python at 100.0%. The 'Suggested Workflows' section shows a workflow for 'SLSA Generic generator'.

CORE
A Plug-and-Play **CON**versational Agent for **RE**commender System

[Project](#) | [Paper](#) | [Documentation](#)

CORE is a plug-and-play conversational agent for any recommender system built upon **PyTorch**. CORE is:

- Comprehensive:** CORE provides a data manager, an offline trainer, and an online checker.
- Flexible:** CORE could be integrated into any recommender system.
- Efficient:** CORE is built upon an online decision tree algorithm instead of heavily learning algorithms.

Conclusion

- CORE only requires **estimated scores** from recommender systems (offline-training) and **APIs** from conversational agent (online checking).
- LLMs can plug-in specific task-related **tools** to encode **user's historical data and tabular data**.

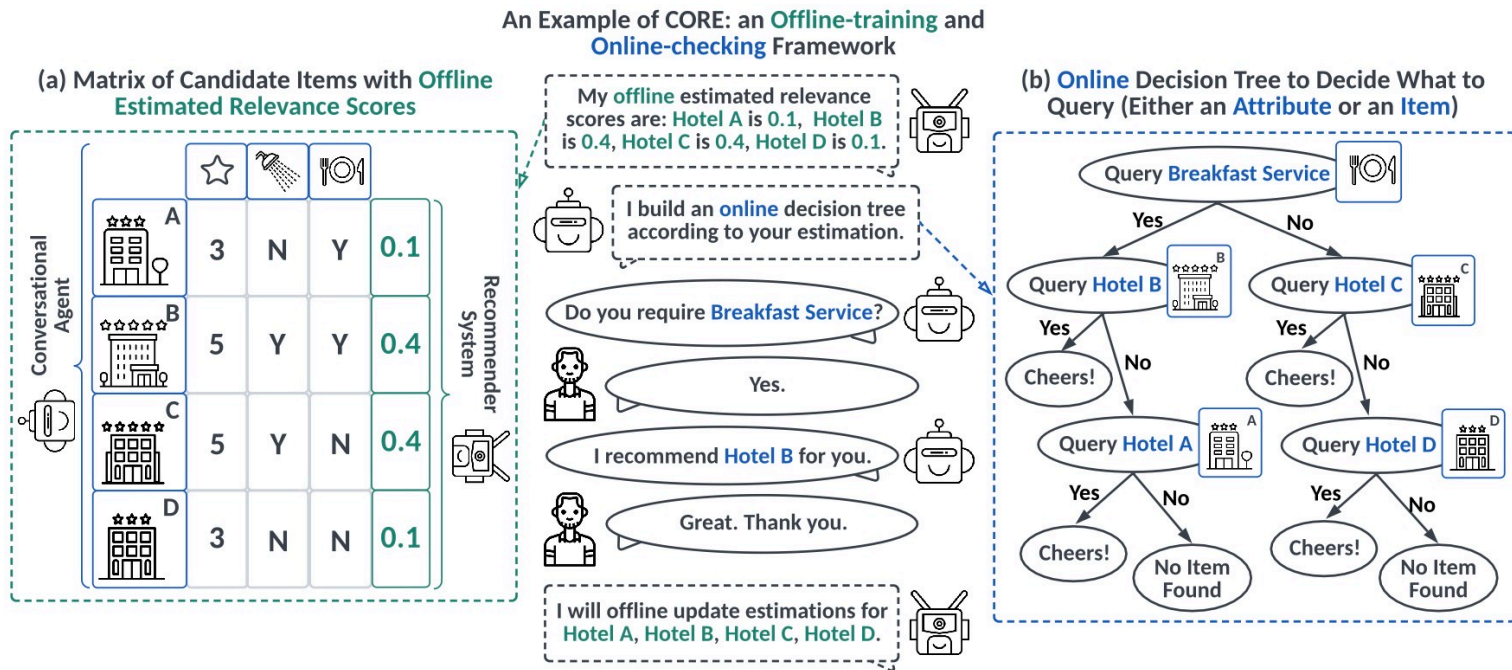


Figure: Overview of our offline-training and online-checking framework.

Thanks for Your Listening



Paper Link



Project Link



Slides Link



Poster Link